
MYSQL ASSIGNMENT

DDL COMMANDS & CONSTRAINTS

Submitted by:
SNEHA MURALI

Contents

DDL Commands:	2
Table Creation (CREATE):	2
Table Alteration (ALTER):	4
Table Renaming (RENAME):	6
Table Truncation (TRUNCATE):	7
Constraints:	8
Database Recreation	8
Departments Table:	8
Location Table:	8
Employees Table:.....	9
Glimpses of Executed SQL Commands	10
Conclusion:	13

DDL Commands:

Table Creation (CREATE):

The following SQL statements has created a database named "employee"

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'employee' database is expanded, showing 'Tables', 'Views', 'Stored Procedures', and 'Functions'. On the right, the SQL Server Enterprise Manager - Query window shows the following SQL statements:

```
2 • create database employee;  
3 • use employee;  
4
```

Below the query window, the Execution Log shows the following results:

Step	Time	Command	Rows Affected
4	07:14:26	create database employee	1 row(s) affected
5	07:14:26	use employee	0 row(s) affected

Created three tables named:

DEPARTMENTS

The screenshot shows the SQL Server Enterprise Manager - Query window with the following SQL statement:

```
create table DEPARTMENTS(DEPARTMENT_ID INT,  
DEPARTMENT_NAME VARCHAR(100));
```

Below the query window, the Result Grid shows the following columns:

DEPARTMENT_ID	DEPARTMENT_NAME
---------------	-----------------

LOCATION

The screenshot shows the SQL Server Enterprise Manager - Query window with the following SQL statement:

```
CREATE TABLE LOCATION(LOCATION_ID INT,  
LOCATION VARCHAR(30));
```

Below the query window, the Result Grid shows the following columns:

LOCATION_ID	LOCATION
-------------	----------

EMPLOYEES

The screenshot shows the SQL Server Enterprise Manager - Query window with the following SQL statement:

```
CREATE TABLE EMPLOYEES(EMPLOYEE_ID INT,  
EMPLOYEE_NAME VARCHAR(50),  
GENDER ENUM ('M', 'F'),  
AGE INT,  
HIRE_DATE DATE,  
DESIGNATION VARCHAR(100),  
DEPARTMENT_ID INT,  
LOCATION_ID INT,  
SALARY DECIMAL(10,2))  
;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Contents:

EMPLOYEE_ID	EMPLOYEE_NAME	GENDER	AGE	HIRE_DATE	DESIGNATION	DEPARTMENT_ID	LOCATION_ID	SALARY
-------------	---------------	--------	-----	-----------	-------------	---------------	-------------	--------

OUTPUT:

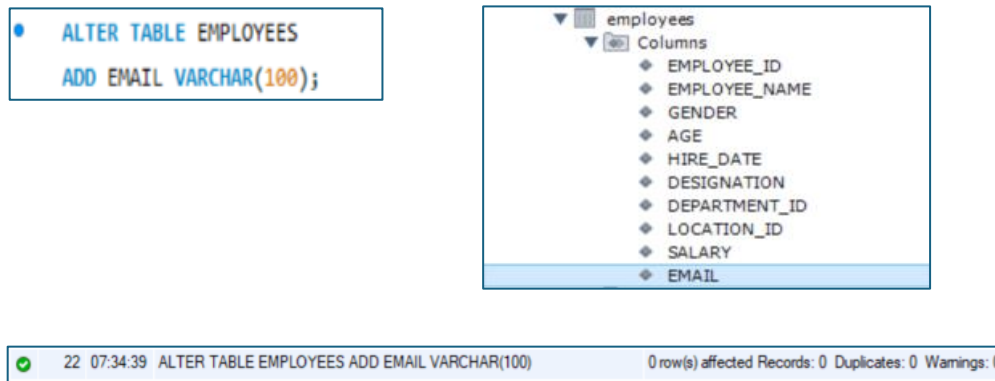
employee
Tables
departments
employees
location

✓	13	07:25:48	create table DEPARTMENTS(DEPARTMENT_ID INT, DEPARTMENT_NAME VAR...	0 row(s) affected
✓	14	07:25:48	CREATE TABLE LOCATION(LOCATION_ID INT, LOCATION VARCHAR(30))	0 row(s) affected
✓	15	07:25:48	CREATE TABLE EMPLOYEES(EMPLOYEE_ID INT, EMPLOYEE_NAME VARCHA...	0 row(s) affected

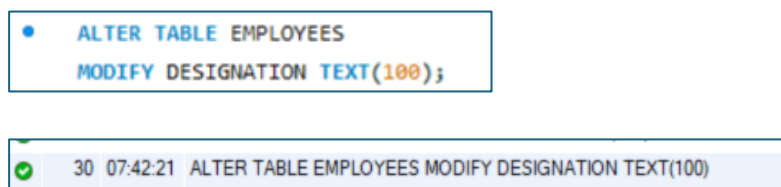
Table Alteration (ALTER):

Considering the scenarios, the following SQL Statements written to alter the structure of the tables accordingly:

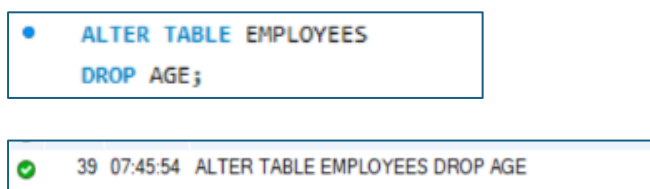
Added a new column named "email" to the Employees table to store employee email addresses.



Modified the data type of the "designation" column in the Employees table to support a wider range of values upto 100.



Dropped the "age" column from the Employees table.



Renamed the "HIRE_DATE" column to "DATE_OF_JOINING".

```
• ALTER TABLE EMPLOYEES  
  RENAME COLUMN HIRE_DATE TO DATE_OF_JOINING;
```

```
✓ 59 07:53:47 ALTER TABLE EMPLOYEES RENAME COLUMN HIRE_DATE TO DAT...
```

OUTPUT:

Table: employees	
Columns:	
EMPLOYEE_ID	int
EMPLOYEE_NAME	varchar(50)
GENDER	enum('M','F')
DATE_OF_JOINING	date
DESIGNATION	text
DEPARTMENT_ID	int
LOCATION_ID	int
SALARY	decimal(10,2)
EMAIL	varchar(100)

Table Renaming (RENAME):

Following are the SQL statements to rename the following tables:

Renamed the "Departments" table to "DEPARTMENTS_INFO".

```
• RENAME TABLE DEPARTMENTS TO DEPARTMENTS_INFO;
```

```
✓ 70 08:04:00 RENAME TABLE DEPARTMENTS TO DEPARTMENTS_INFO
```

Renamed the "Location" table to "Locations".

```
• RENAME TABLE LOCATION TO LOCATIONS;
```

```
✓ 82 08:07:04 RENAME TABLE LOCATION TO LOCATIONS
```

Table Truncation (TRUNCATE):

Written an SQL statement to truncate the Employees table.

• `TRUNCATE TABLE EMPLOYEES;`

✓ 95 08:10:09 TRUNCATE TABLE EMPLOYEES

Database & Table Dropping (DROP):

Written the SQL statements to drop;

The Employees table:

• `DROP TABLE EMPLOYEES;`



✓ 109 08:13:00 DROP TABLE EMPLOYEES

The "employee" database:

• `DROP DATABASE EMPLOYEE;`

✓ 124 08:16:12 DROP DATABASE EMPLOYEE

Constraints:

Database Recreation:

Recreated the employee database and tables using the provided schema, ensuring that all tables are created with the appropriate constraints as instructed.

```
• CREATE DATABASE EMPLOYEE;  
• USE EMPLOYEE;
```



Departments Table:

```
• CREATE table DEPARTMENTS (DEPARTMENT_ID INT PRIMARY KEY,  
  DEPARTMENT_NAME VARCHAR(100) NOT NULL UNIQUE);
```

Added PRIMARY KEY as constraint that "department_id" uniquely identifies each department.

Added UNIQUE and NOT NULL constraints on the "department_name" to avoid duplicate and null entries.

Location Table:

```
• CREATE TABLE LOCATION (LOCATION_ID INT PRIMARY KEY AUTO_INCREMENT,  
  LOCATION VARCHAR(30) NOT NULL UNIQUE);
```

Added PRIMARY KEY and AUTO_INCREMENT as a mechanism to automatically generate unique identifiers for each location, ensuring that they are incremented sequentially.

Added NOT NULL and UNIQUE constraints to prevent the insertion of null and duplicate locations.

Employees Table:

Added PRIMARY KEY as constraint guaranteeing that each employee has a distinct identifier.

```
CREATE TABLE EMPLOYEES(EMPLOYEE_ID INT PRIMARY KEY,
```

Created a restriction by adding NOT NULL as a constraint to ensure that the employee's name is always provided.

```
EMPLOYEE_NAME VARCHAR(50) NOT NULL,
```

Limited the acceptable values for the "gender" field to only 'M' or 'F' by adding CHECK constraint.

```
GENDER CHAR(1) CHECK(GENDER IN ('M','F')) ,
```

Enforce a condition to ensure that the employee's age is 18 or above.

```
AGE INT CHECK (AGE>=18),
```

Automatically assign the current date to the "HIRE_DATE" field if not specified.

```
HIRE_DATE DATE DEFAULT (CURRENT_DATE),
```

Establish links between the "DEPARTMENT_ID" and "LOCATION_ID" fields in the "employees" table and their respective tables.

```
FOREIGN KEY (DEPARTMENT_ID) REFERENCES DEPARTMENTS(DEPARTMENT_ID),  
FOREIGN KEY (LOCATION_ID) REFERENCES LOCATION(LOCATION_ID));
```

Glimpses of Executed SQL Commands

```
drop database if exists employee;
```

```
create database employee;
```

```
use employee;
```

```
create table DEPARTMENTS (DEPARTMENT_ID INT,
```

```
DEPARTMENT_NAME VARCHAR (100)
```

```
)
```

```
;
```

```
CREATE TABLE LOCATION (LOCATION_ID INT,
```

```
LOCATION VARCHAR (30)
```

```
)
```

```
;
```

```
CREATE TABLE EMPLOYEES (EMPLOYEE_ID INT,
```

```
EMPLOYEE_NAME VARCHAR (50),
```

```
GENDER ENUM ('M','F'),
```

```
AGE INT,
```

```
HIRE_DATE DATE,
```

```
DESIGNATION VARCHAR (100),
```

```
DEPARTMENT_ID INT,
```

```
LOCATION_ID INT,
```

```
SALARY DECIMAL (10,2)
```

```
)
```

```
;
```

```
SELECT * FROM DEPARTMENTS;
```

```
SELECT * FROM LOCATION;
```

```
SELECT * FROM EMPLOYEES;
```

```

ALTER TABLE EMPLOYEES
ADD EMAIL VARCHAR (100);
ALTER TABLE EMPLOYEES
MODIFY DESIGNATION TEXT (100);
ALTER TABLE EMPLOYEES
DROP AGE;
ALTER TABLE EMPLOYEES
RENAME COLUMN HIRE_DATE TO DATE_OF_JOINING;
RENAME TABLE DEPARTMENTS TO DEPARTMENTS_INFO;
RENAME TABLE LOCATION TO LOCATIONS;
TRUNCATE TABLE EMPLOYEES;
DROP TABLE EMPLOYEES;
DROP DATABASE EMPLOYEE;
create database employee;
use employee;
CREATE table DEPARTMENTS (DEPARTMENT_ID INT PRIMARY KEY,
DEPARTMENT_NAME VARCHAR (100) NOT NULL UNIQUE);
CREATE TABLE LOCATION (LOCATION_ID INT PRIMARY KEY AUTO_INCREMENT,
LOCATION VARCHAR (30) NOT NULL UNIQUE);
CREATE TABLE EMPLOYEES (EMPLOYEE_ID INT PRIMARY KEY,
EMPLOYEE_NAME VARCHAR (50) NOT NULL,
GENDER CHAR (1) CHECK (GENDER IN ('M','F')),
AGE INT CHECK (AGE >= 18),
HIRE_DATE DATE DEFAULT (CURRENT_DATE),
DESIGNATION TEXT (100),
DEPARTMENT_ID INT,

```

```
LOCATION_ID INT,  
SALARY DECIMAL (10,2),  
EMAIL VARCHAR (100),  
FOREIGN KEY (DEPARTMENT_ID) REFERENCES DEPARTMENTS(DEPARTMENT_ID),  
FOREIGN KEY (LOCATION_ID) REFERENCES LOCATION(LOCATION_ID)  
)  
;
```

The drop database if exists employee; command was added to ensure that any previously created version of the database is removed before recreating it.

Conclusion:

This assignment provided a practical understanding of MySQL DDL commands and constraints by designing and managing an EMPLOYEE DATABASE. Tasks such as create, alter, rename, truncate, and drop tables helps to define and modify the organizational needs. The constraints applied here are- primary keys, unique values, auto-increment, check conditions, default values, and foreign key relationships. Overall, the assignment reinforced the importance of DDL commands in building database schemas and highlighted how constraints safeguard against invalid or duplicate entries, making the database both efficient and dependable.